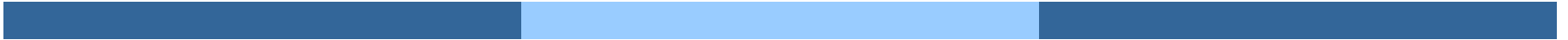


File-System Interface



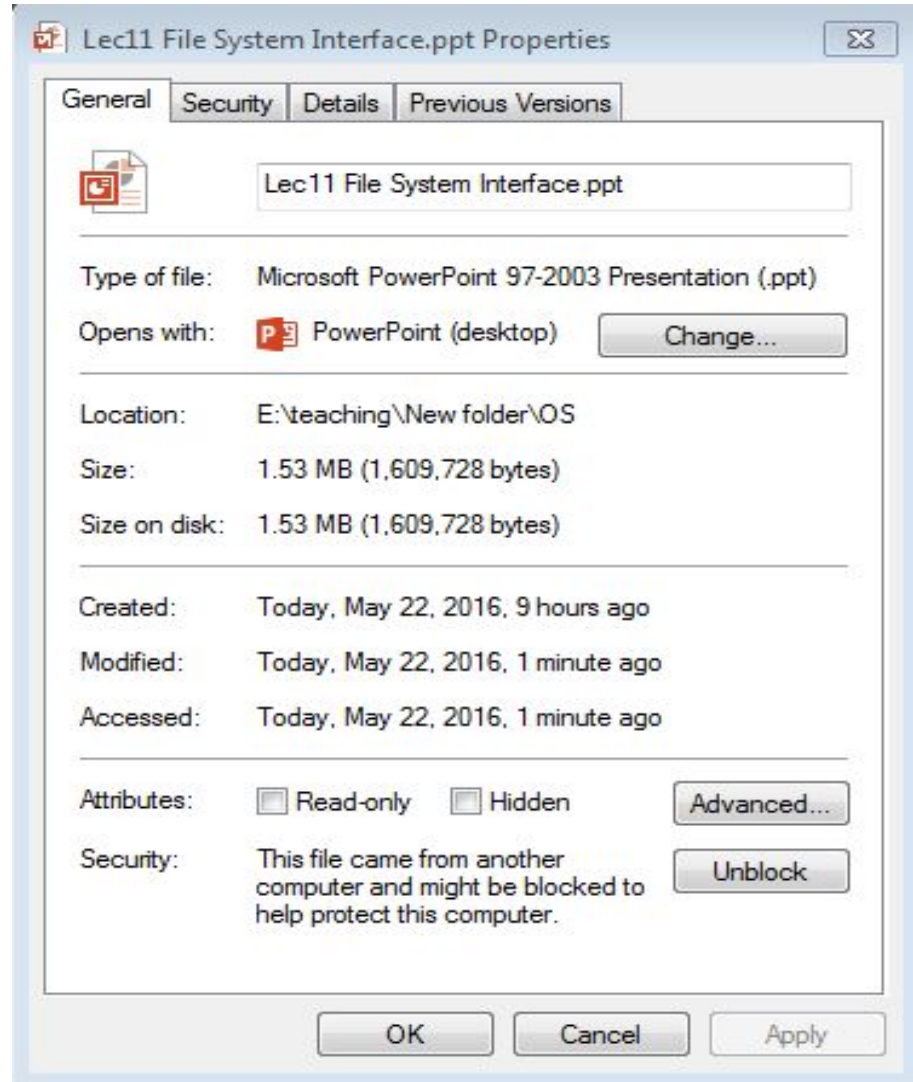
File Concept

- Computers can store information on various storage media
 - operating system **abstracts** from the physical properties of its storage devices to define a **logical storage unit**, the **file**
 - Smallest unit of logical secondary storage
- Contiguous logical address space
- Types:
 - Data
 - 4 numeric, alphanumeric, binary
 - Program
 - 4 Source and object forms
- Contents defined by file's creator
 - Many types (even free form or formatted)
 - 4 File structure depends on file type: Consider **text file**, **source file**, **executable file**
- Independent of the creating process, or even operating system

File Attributes

- **Name** – only information kept in human-readable form
 - When a file is named, it becomes **independent of the process**, the user, and even the system that created it.
- **Identifier** – unique tag (number) identifies file within file system
- **Type** – needed for systems that support different types
- **Location** – pointer to file location on device
- **Size** – current file size
- **Protection** – controls who can do reading, writing, executing
- **Time, date, and user identification** – data for protection, security, and usage monitoring
- Many variations, including **extended file attributes such as file checksum**
- Information about files are kept in the **directory structure**, which is maintained on the disk

File info Window on Mac OS X, Windows



File Operations

- File is an **abstract data type**
 - The operating system provides system calls to perform different operations on file
- **Create** – allocate space in file system, [create directory entry](#)
- **Write** – at [write pointer](#) location
- **Read** – at [read pointer](#) location
- **Reposition within file** – reposition the pointer : [seek](#)
- **Delete** – erase directory entry, release file space
- **Truncate** – delete file contents, but keep attributes
- ***Open(F_i)*** – search the directory structure on disk for entry F_i , and move the content of entry to memory
- ***Close (F_i)*** – move the content of entry F_i in memory to directory structure on disk

Open Files

- Many file operations involve searching the directory for the entry associated with the named file.
 - **Open** system call made necessary to avoid this searching (some systems implicitly open a file on first reference)
 - Indexed search to open-file table, avoids directory searches
 - **Create** and **delete** system calls work with closed files
- Several pieces of data are needed to manage open files:
 - **Open-file table**: tracks open files
 - File pointer: pointer to last read/write location, **per process** that has the file open
 - several pieces of information are associated with an open file: File pointer, **file-open count**, disk location of file, access rights etc
 - **File-open count**: counter of number of times a file is open – to allow removal of data from open-file table when last processes closes it
 - Disk location of the file: cache of data access information
 - Access rights: **per-process access mode** information

Open File Locking

- Provided by **some operating systems** and file systems
 - Locking an open file, or sections of a file
 - Similar to reader-writer locks
 - **Shared lock** similar to reader lock – several processes can acquire concurrently
 - **Exclusive lock** similar to writer lock
- Mediates access to a file
- Mandatory or advisory:
 - **Mandatory** – access is denied depending on locks held and requested
assume a process acquires an **exclusive lock on system.log** file. If we attempt to open it from another process—for example, a text editor—the operating system will prevent access until the exclusive lock is released
 - **Advisory** – processes can find status of locks and decide what to do
the operating system will not prevent the text editor from acquiring access to **system.log**. Rather, the text editor must be written so that it manually acquires the lock before accessing the file

File Locking Example – Java API

```
import java.io.*;
import java.nio.channels.*;
public class LockingExample {
    public static final boolean EXCLUSIVE = false;
    public static final boolean SHARED = true;
    public static void main(String arsg[]) throws IOException {
        FileLock sharedLock = null;
        FileLock exclusiveLock = null;
        try {
            RandomAccessFile raf = new RandomAccessFile("file.txt", "rw");
            // get the channel for the file
            FileChannel ch = raf.getChannel();
            // this locks the first half of the file - exclusive
            exclusiveLock = ch.lock(0, raf.length()/2, EXCLUSIVE);
            /** Now modify the data . . . */
            // release the lock
            exclusiveLock.release();
        }
    }
}
```

File Locking Example – Java API (Cont.)

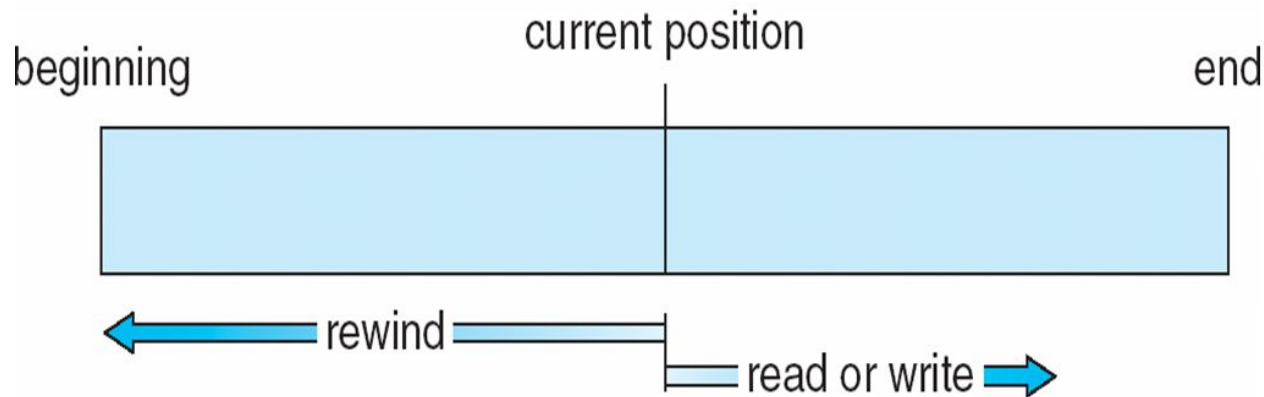
```
        // this locks the second half of the file - shared
        sharedLock = ch.lock(raf.length()/2+1, raf.length(),
SHARED);
        /** Now read the data . . . */
        // release the lock
        sharedLock.release();
    } catch (java.io.IOException ioe) {
        System.err.println(ioe);
    }finally {
        if (exclusiveLock != null)
            exclusiveLock.release();
        if (sharedLock != null)
            sharedLock.release();
    }
}
```

File Structure

- File types also can be used to indicate the internal structure of the file
 - Understandable by other programs, operating system or users etc.
 - Files must conform to the required structure
 - Some OS introduce **set of system supported file structures** with set of special operations for manipulating files with those structures
 - Disadvantages: OS **size**, adding **new file structures** is difficult
- Some OS impose and support **minimal number of file system** (adopted in UNIX, windows and many others)
 - UNIX file: sequence of 8-bit bytes (No interpretation of these bits made by the OS)
 - 4 Flexible with little support. Interpretation code needed in application programs
- All OS must **support at-least one structure**: executable files
 - Enables to load and run these programs

Internal File Structure

- Internally, locating an offset within a file can be complicated for the operating system
 - Disk I/O in units of one block
 - Block sizes won't match the length of desired logical record
 - 4 Pack number of logical records into physical blocks
- UNIX files are simply streams of bytes, with each byte addressable with its offset from beginning or end of file
 - Logical record size is one byte: automatically packed into physical disk blocks
- Some portion from last block of each file is usually wasted
 - All file systems suffer from internal fragmentation



Access Methods

- **Sequential Access:** Simplest solution. Information retrieved in order, on record one after the other

```
read next
write next
reset
```

- **Direct Access** – file composed of fixed length **logical records** (Numbered sequence of blocks). Allows rapid reading or writing of records with no order

```
read n      (instead of read next)
write n (instead of write next)
position to n (Other Choice)
    read next
    write next
rewrite n
```

n = **relative block number**

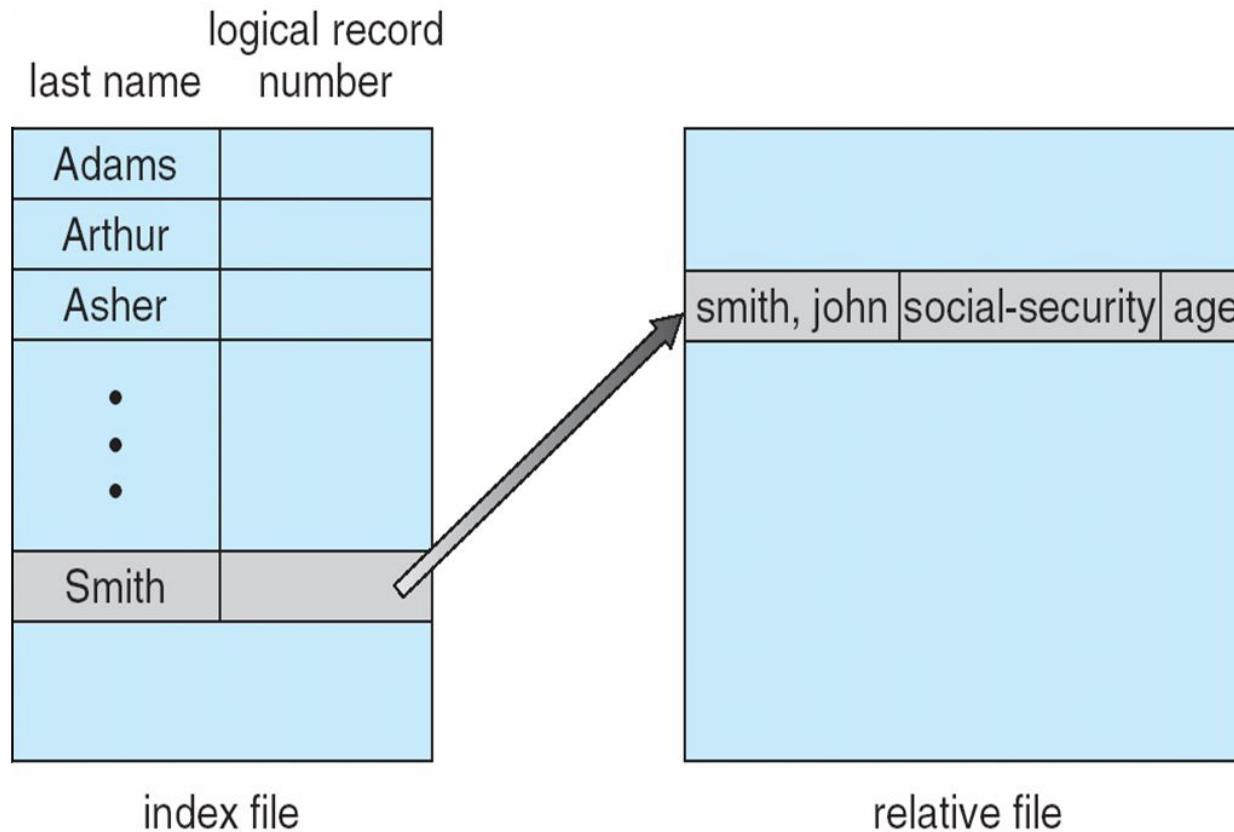
- Relative block numbers allow OS to decide where file pointer should be placed
 - Called **allocation problem**

sequential access	implementation for direct access
<i>reset</i>	<i>cp</i> = 0;
<i>read next</i>	<i>read cp</i> ; <i>cp</i> = <i>cp</i> + 1;
<i>write next</i>	<i>write cp</i> ; <i>cp</i> = <i>cp</i> + 1;

Other Access Methods

- Can be built on top of base methods
- General involve creation of an [index](#) for the file
- Keep index in memory for fast determination of location of data to be operated on (consider UPC code plus record of data about that item)
- If too large, index (in memory) of the index (on disk)
- IBM indexed sequential-access method (ISAM)
 - Small master index, points to disk blocks of secondary index
 - File kept sorted on a defined key
 - All done by the OS
- VMS operating system provides index and relative files as another example (see next slide)

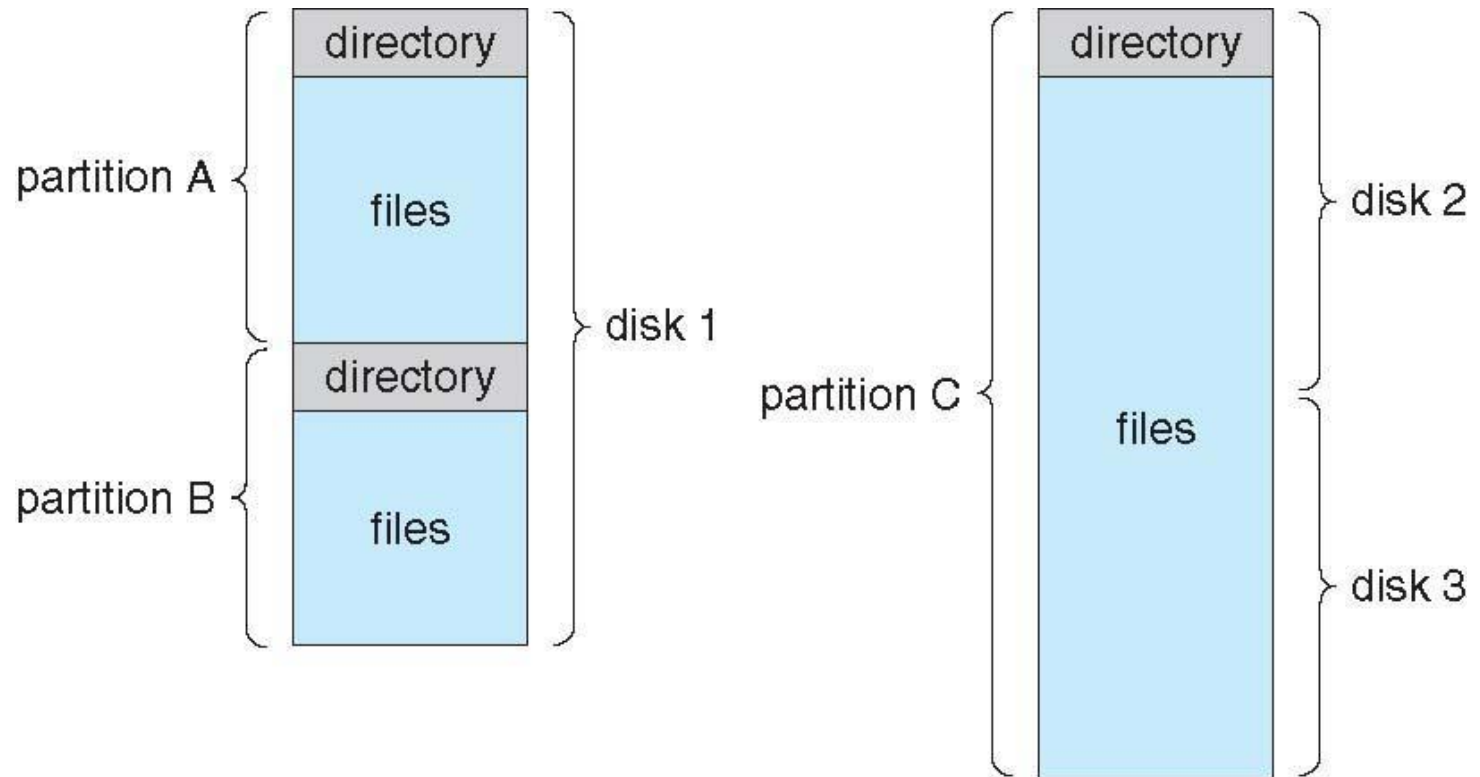
Example of Index and Relative Files



Directory Structure

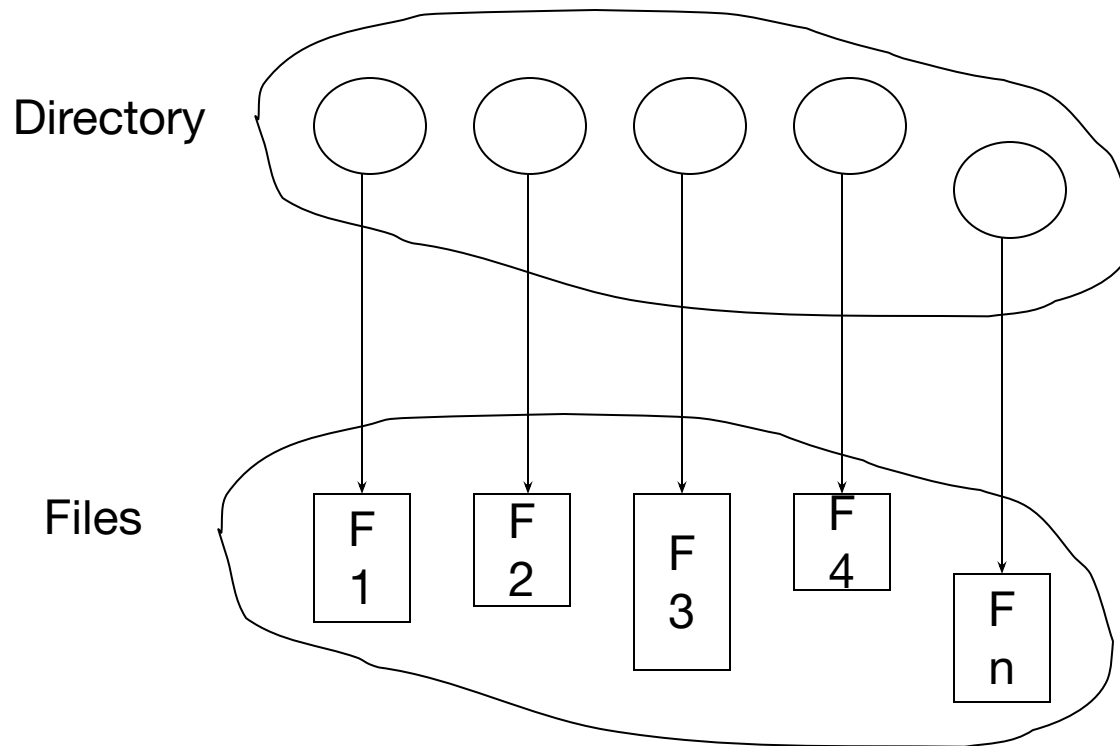
- Millions of files within a general purpose computer
 - Stored on random access storage devices
 - Storage devices having a single file system or partitioned to carry multiple file systems (known as volumes)
 - 4 Limit the sizes of individual file systems
- Each volume that contains a file system must also contain information about the files in the system
 - Device directory or **volume table of contents**
 - 4 Name
 - 4 Location
 - 4 Size
 - 4 Type
 - 4
- File system can be extensive.
 - Useful to organize files into directories

A Typical File-system Organization



Single Level Directory

- A collection of nodes containing information about all files



Both the directory structure and the files reside on disk

Two Level Directory

In the two-level directory structure, each user has his own **user file directory (UFD)**. The UFDs have similar structures, but each lists only the files of a single user. When a user job starts or a user logs in, the system's **master file directory (MFD)** is searched. The MFD is indexed by user name or account number, and each entry points to the UFD for that user

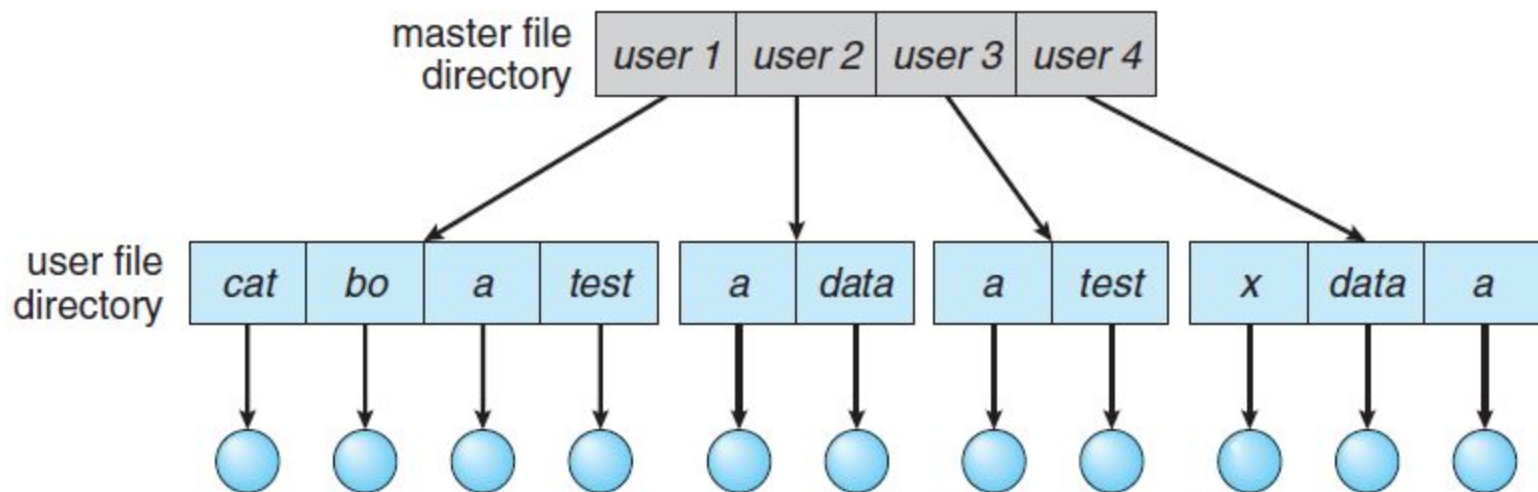
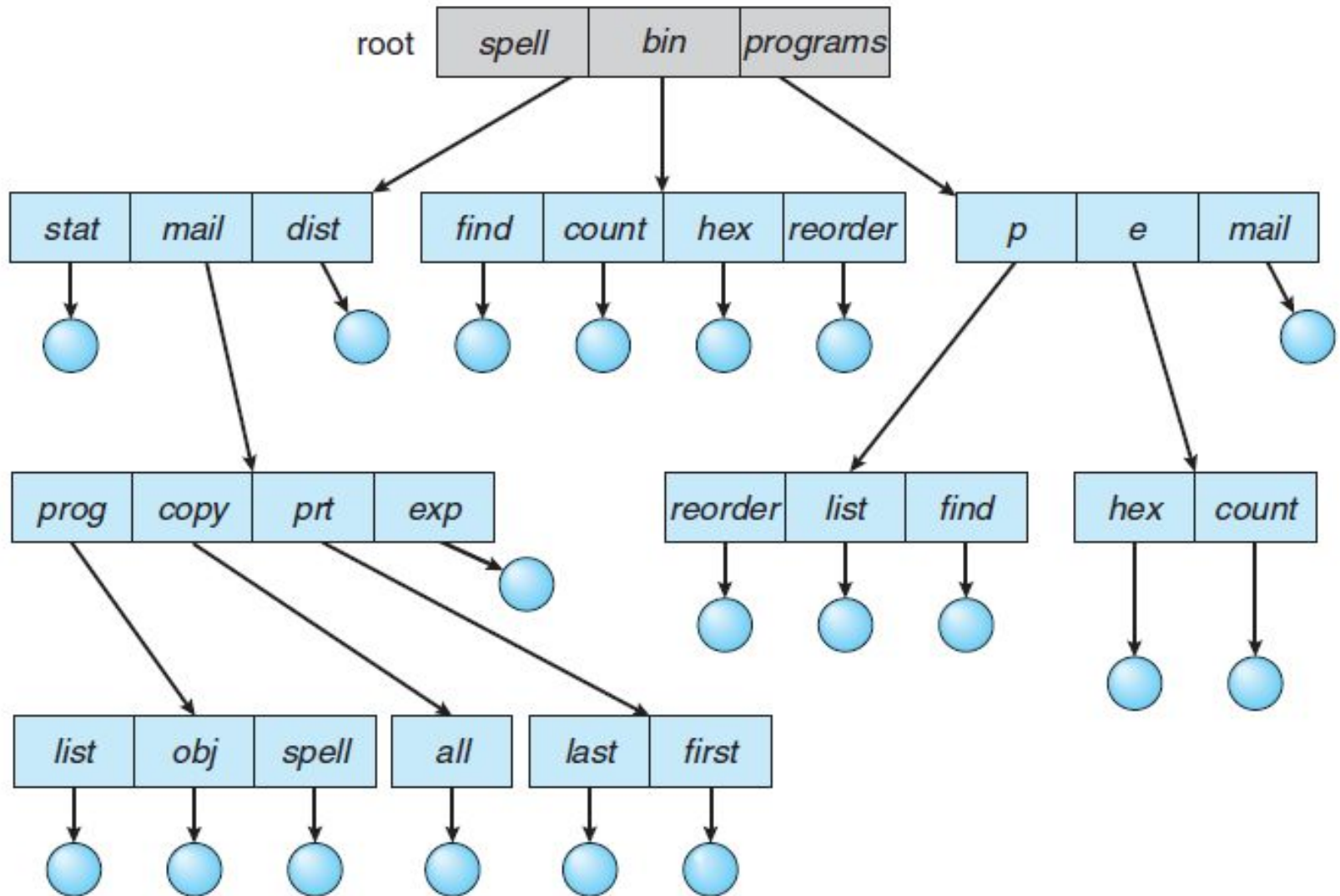


Figure 11.10 Two-level directory structure.

Tree Structured Directories



Tree-Structured Directories (Cont.)

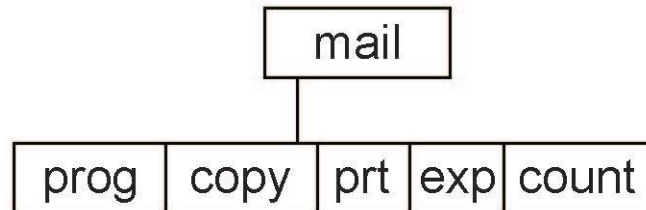
- Efficient searching
- Grouping Capability
- Current directory (**working directory**)
 - `cd /spell/mail/prog`
 - `type list`

Tree-Structured Directories (Cont)

- **Absolute** or **relative** path name
- Creating a new file is done in current directory
- Delete a file
`rm <file-name>`
- Creating a new subdirectory is done in current directory
`mkdir <dir-name>`

Example: if in current directory `/mail`

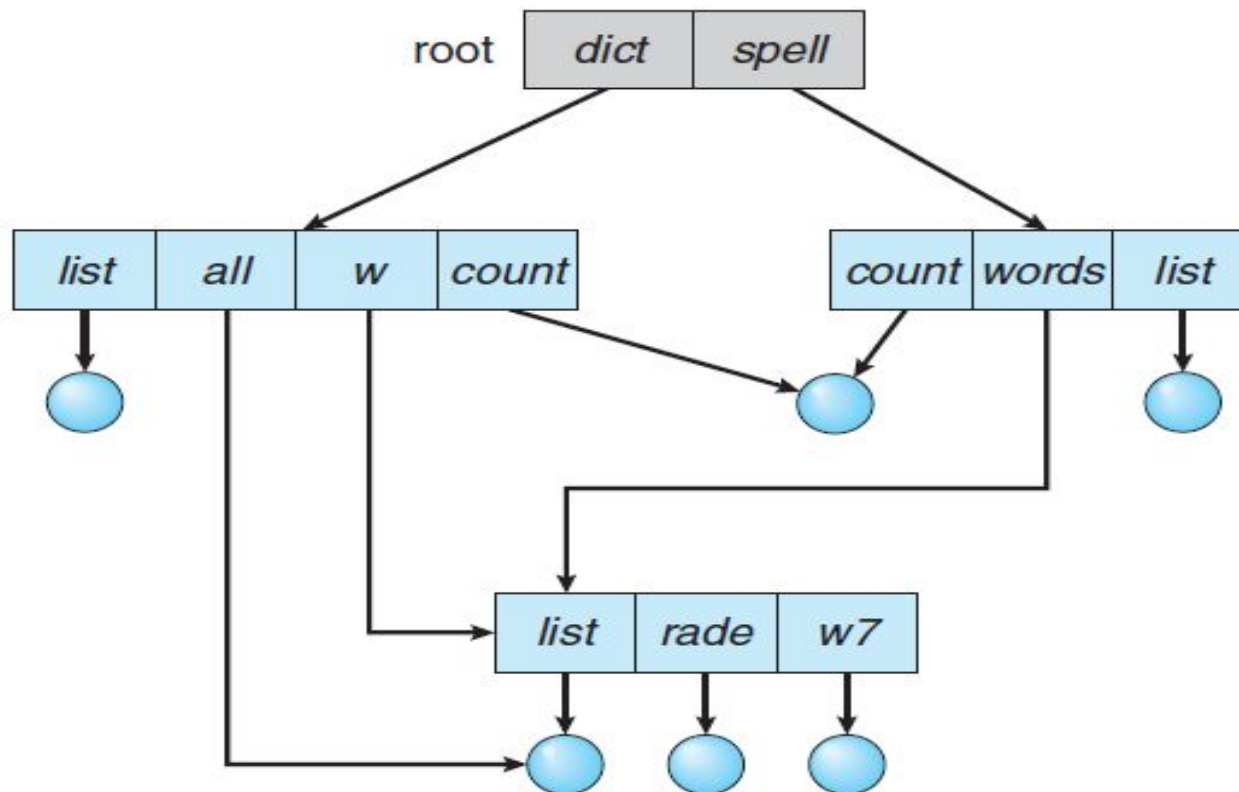
`mkdir count`



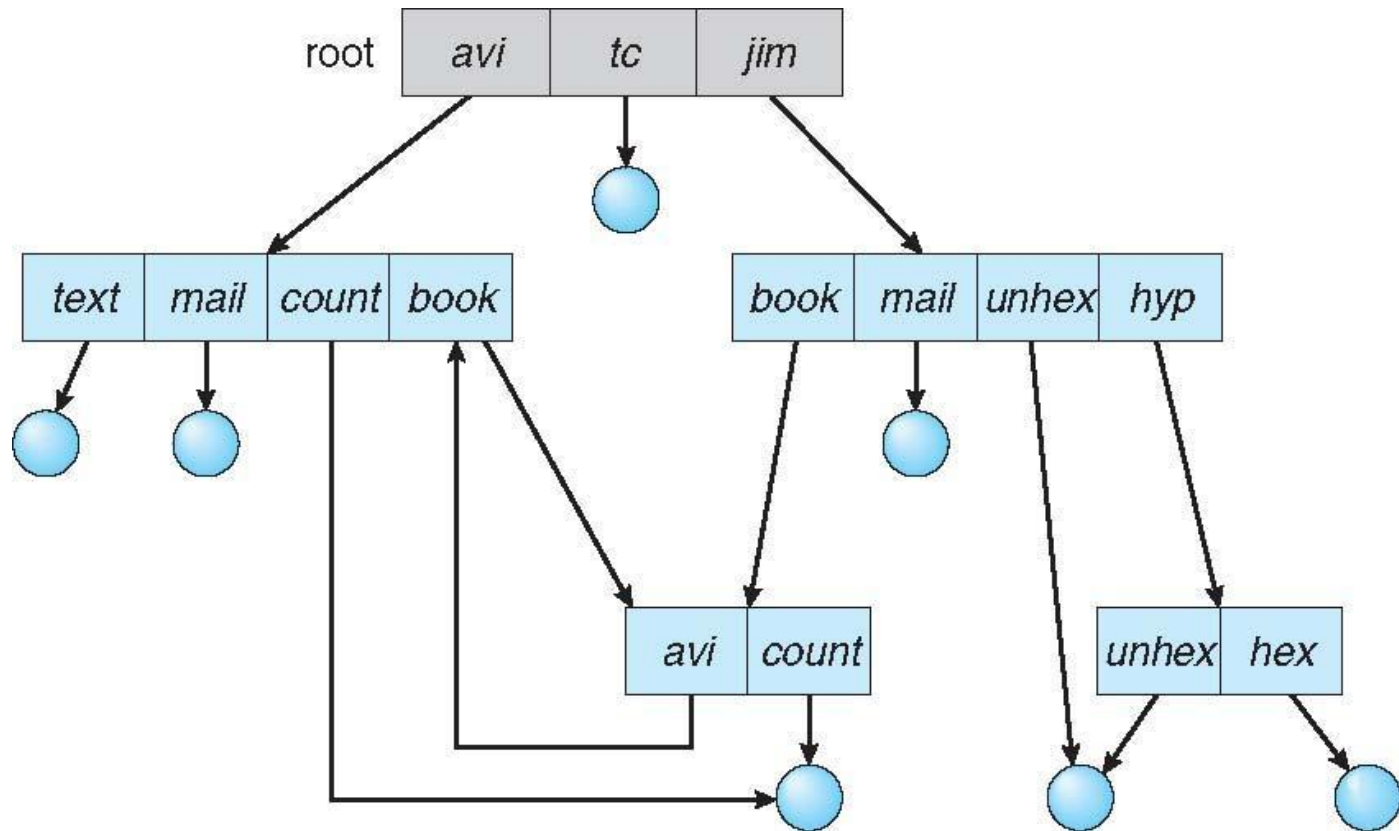
Deleting “mail” ⇒ deleting the entire subtree rooted by “mail”

Acyclic-Graph Directories

Consider two programmers who are working on a joint project. The files associated with that project can be stored in a subdirectory, separating them from other projects and files of the two programmers. But since both programmers are equally responsible for the project, both want the subdirectory to be in their own directories. In this situation, the common subdirectory should be shared. A shared directory or file exists in the file system in two (or more) places at once.



General Graph Directory



Types of File Systems

- We mostly talk of general-purpose file systems
- But systems frequently have many file systems, some general- and some special- purpose
- Consider Solaris has
 - tmpfs – memory-based volatile FS for fast, temporary I/O
 - objfs – interface into kernel memory to get kernel symbols for debugging
 - ctfs – contract file system for managing daemons
 - lofs – loopback file system allows one FS to be accessed in place of another
 - procfs – kernel interface to process structures
 - ufs, zfs – general purpose file systems

Operations Performed on Directory

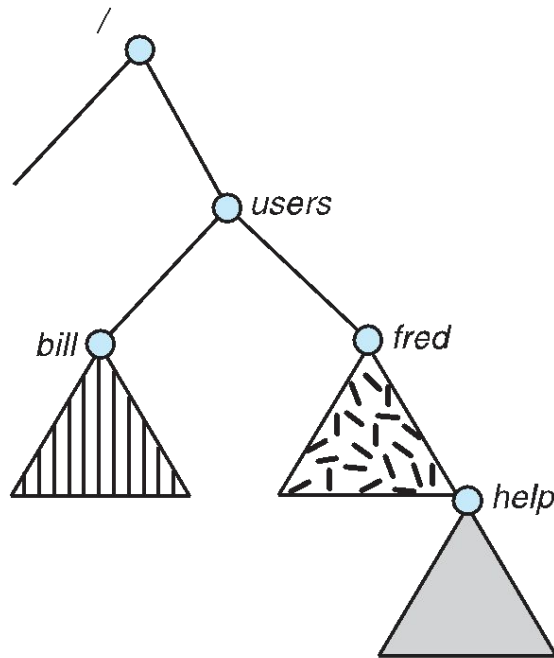
- Search for a file
- Create a file
- Delete a file
- List a directory
- Rename a file
- Traverse the file system

General Graph Directory (Cont.)

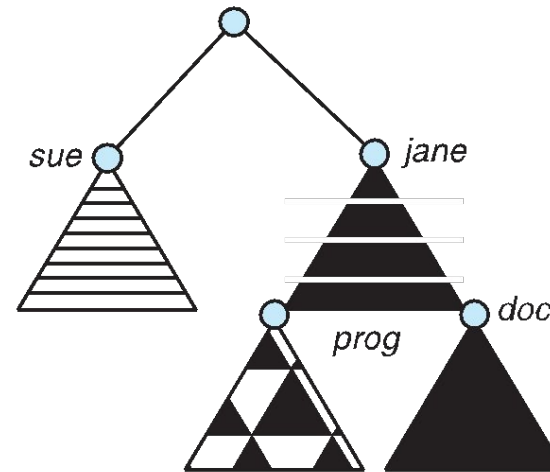
- How do we guarantee no cycles?
 - Allow only links to file not subdirectories
 - **Garbage collection**
 - Every time a new link is added use a cycle detection algorithm to determine whether it is OK

File System Mounting

- A file system must be **mounted** before it can be accessed
- A unmounted file system (i.e., Fig. 11-11(b)) is mounted at a **mount point**

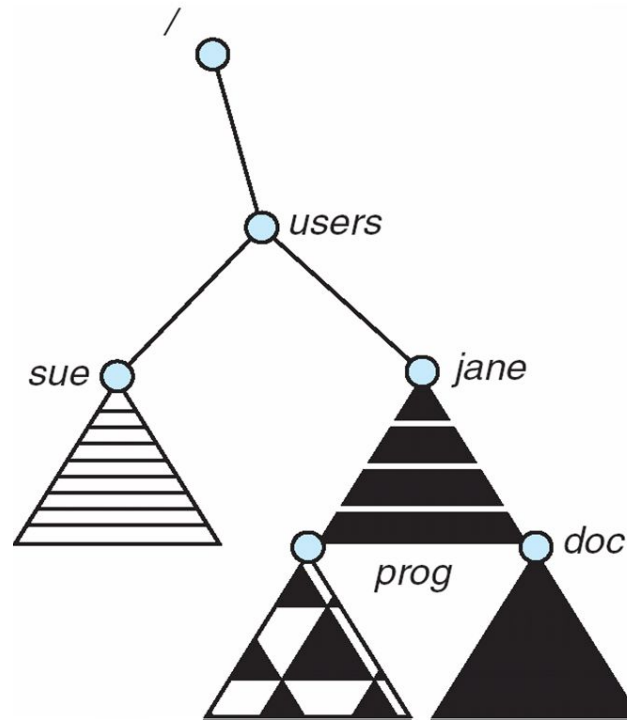


(a)



(b)

Mount Point



File Sharing

- Sharing of files on multi-user systems is desirable
- Sharing may be done through a protection scheme
- If multi-user system
 - User IDs identify users, allowing permissions and protections to be per-user
 - Group IDs allow users to be in groups, permitting group access rights
 - Owner of a file / directory
 - Group of a file / directory
- On distributed systems, files may be shared across a network
- Network File System (NFS) is a common distributed file-sharing method

File Sharing – Remote File Systems

- Uses networking to allow file system access between systems
 - Manually via programs like **FTP**
 - Automatically, seamlessly using **distributed file systems**
 - Semi automatically via the **world wide web**
- **Client-server** model allows clients to mount remote file systems from servers
 - Server can serve multiple clients
 - Client and user-on-client identification is insecure or complicated
 - **NFS** is standard UNIX client-server file sharing protocol
 - **CIFS** is standard Windows protocol
 - Standard operating system file calls are translated into remote calls
- Distributed Information Systems (**distributed naming services**) such as LDAP, DNS, NIS, Active Directory implement unified access to information needed for remote computing

File Sharing – Failure Modes

- All file systems have failure modes
 - For example corruption of directory structures or other non-user data, called **metadata**
- Remote file systems add new failure modes, due to network failure, server failure
- Recovery from failure can involve **state information** about status of each remote request
- **Stateless** protocols such as NFS v3 include all information in each request, allowing easy recovery but less security

File Sharing – Consistency Semantics

- Specify how multiple users are to access a shared file simultaneously
 - Similar to Ch 5 process synchronization algorithms
 - 4 Tend to be less complex due to disk I/O and network latency (for remote file systems)
 - Andrew File System (AFS) implemented complex remote file sharing semantics
 - Unix file system (UFS) implements:
 - 4 Writes to an open file visible immediately to other users of the same open file
 - 4 Sharing file pointer to allow multiple users to read and write concurrently
 - AFS has session semantics
 - 4 Writes only visible to sessions starting after the file is closed

Protection

- File owner/creator should be able to control:
 - what can be done
 - by whom
- Types of access
 - **Read**
 - **Write**
 - **Execute**
 - **Append**
 - **Delete**
 - **List**

Access Lists and Groups

- Mode of access: read, write, execute
- Three classes of users on Unix / Linux

RWX

a) **owner access** 7 $\Rightarrow$ 1 1 1

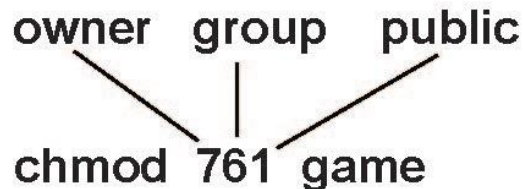
RWX

b) **group access** 6 $\Rightarrow$ 1 1 0

RWX

c) **public access** 1 $\Rightarrow$ 0 0 1

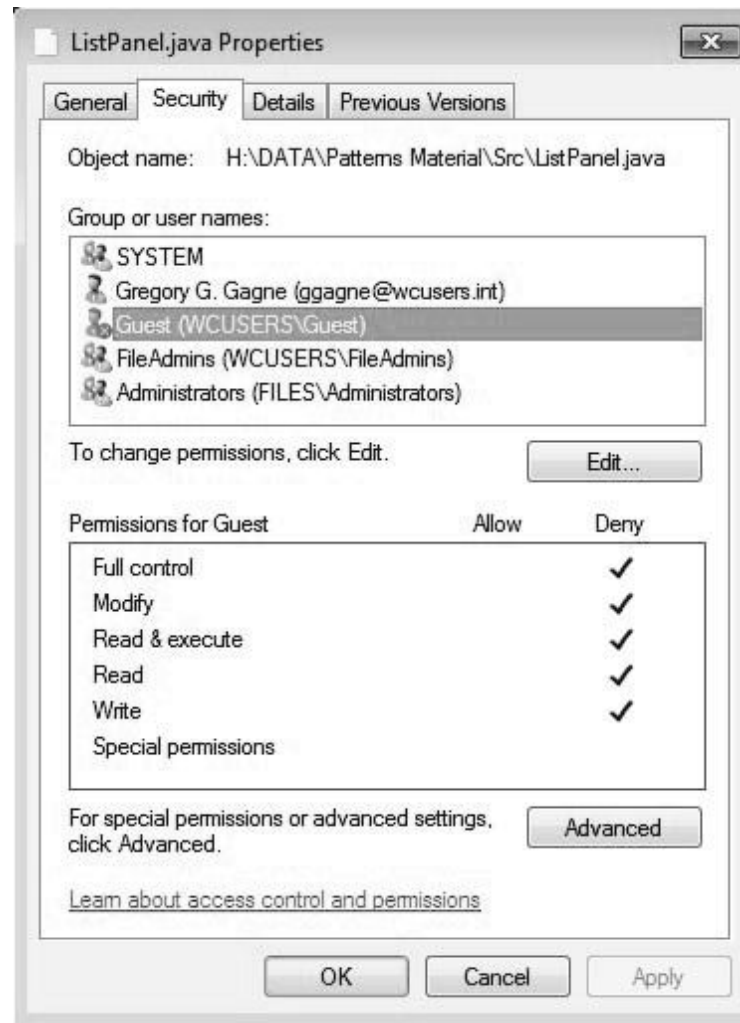
- Ask manager to create a group (unique name), say G, and add some users to the group.
- For a particular file (say *game*) or subdirectory, define an appropriate access.



Attach a group to a file

chgrp G game

Windows 7 Access-Control List Management



A Sample UNIX Directory Listing

-rw-rw-r--	1 pbg	staff	31200	Sep 3 08:30	intro.ps
drwx-----	5 pbg	staff	512	Jul 8 09:33	private/
drwxrwxr-x	2 pbg	staff	512	Jul 8 09:35	doc/
drwxrwx---	2 pbg	student	512	Aug 3 14:13	student-proj/
-rw-r--r--	1 pbg	staff	9423	Feb 24 2003	program.c
-rwxr-xr-x	1 pbg	staff	20471	Feb 24 2003	program
drwx--x--x	4 pbg	faculty	512	Jul 31 10:31	lib/
drwx-----	3 pbg	staff	1024	Aug 29 06:52	mail/
drwxrwxrwx	3 pbg	staff	512	Jul 8 09:35	test/